PROJECT J.A.R.V.I.S.

Complete Product Requirements, Architecture, Implementation Roadmap and Acceptance Plan

Version 1.0 | 28 September 2026 | Planning baseline

Mission: build a voice-first, royal-butler-style AI chief of staff capable of understanding accented English, using authorized computer and online tools, managing communication and projects, and completing verifiable real-world tasks. The product is an evolving software system, not a claim to recreate the fictional Iron Man character.

Planning principle: release useful vertical slices, measure reliability, and add capabilities only after the previous slice works. All external actions are subject to user-defined permissions and relevant platform policies.

1. Executive summary and scope

JARVIS is a desktop-first AI agent with a voice interface, tool orchestration engine, persistent memory, productivity features, developer assistance, and approved communications across supported channels. It will eventually support a mobile companion, advanced telephony, and optional Tunisian Darija. The initial target is a single-user Windows desktop system.

Product outcomes

- Respond naturally to accented spoken English and maintain a distinctive, configurable British-butler personality.
- Execute permitted desktop and browser tasks, produce artifacts, and verify results rather than claiming success without evidence.
- Support development workflows, correspondence, content scheduling, project tracking, research, and daily briefings.
- Maintain searchable, user-controlled memory and activity logs; offer transparent approvals, revocation, and deletion.
- Grow into multilingual and telephone capabilities only after safety, cost, latency, and platform compliance are tested.

Non-goals for the first release

- Training a foundation model from scratch.
- Unsupervised access to every application or social-media account.
- Bypassing CAPTCHA, platform restrictions, rate limits, or access controls.
- Impersonating a human during calls or secretly recording conversations.
- Perfect Darija recognition or universal support for every social network at launch.

2. Complete feature inventory - all 25 selected requirements

The prior feature-selection interface contained 26 individual requirements when the optional Darija capability is counted separately. All are included here; optional does not mean forgotten.

ID	Requirement	Phase
F01	Accented English speech recognition	P0
F02	Optional Tunisian Darija comprehension	P3
F03	Jarvis wake word	P1
F04	Natural interruption / barge-in	P1
F05	Distinctive British-butler voice	P0
F06	Royal titles, dry humor, customizable personality	P0
F07	Professional, casual and silent modes	P0
F08	Open applications and manage windows	P1

ID	Requirement	Phase
F09	Search, organize and summarize local files	P1
F10	Multi-step browser automation	P2
F11	Create/edit documents and spreadsheets	P2
F12	Read code, run tests and investigate errors	P2
F13	GitHub issues and draft pull requests	P2
F14	Monitor long-running development tasks	P2
F15	Read, summarize and draft emails	P2
F16	Send approved messages via connected platforms	P2
F17	Supported WhatsApp, Telegram and Instagram accounts	P3
F18	Prepare and schedule social content	P2
F19	Contact-specific conversation history	P2
F20	Authorized disclosed business calls	P3
F21	Call transcripts and follow-up summaries	P3
F22	Approved long-term preferences	P1
F23	Track projects, deadlines and commitments	P2
F24	Research with source citations	P2
F25	Searchable completed-task history	P1
F26	Schedule-based next-action suggestions	P2

3. Additional supporting requirements

- Reliable voice activity detection, noise handling, microphone permissions, device selection and latency monitoring.
- Tool schemas, execution permissions, dry-run mode, retries, idempotency and structured outcomes.
- Persistent user preferences, project memory, contact memory, retrieval, correction and deletion.
- Task queue, background workers, resumable workflows, cancellation, notifications and audit trail.
- Credentials stored in an OS keychain or secure secrets manager; OAuth where available.
- Accessibility, keyboard/text fallback, visual task dashboard and offline/degraded-mode indicators.
- Automated evaluations, cost tracking, observability, backups, migration strategy and incident recovery.
- Per-platform capability checks: APIs and account types determine which social features are actually available.

4. Representative end-to-end user journeys

Morning briefing

Wake or launch JARVIS; retrieve authorized calendar and tasks; summarize deadlines and email; speak a concise briefing; ask before scheduling or sending.

Desktop action

Hear "Open my development project"; resolve known project location; ask for approval if needed; open the approved application; verify process/window state; report outcome.

Developer workflow

Select repository; read relevant files; reproduce error in isolated workspace; propose fix; run tests; draft GitHub issue or PR; await approval for writes/push.

Internship outreach

Research companies using permitted sources; cite findings; draft personalized emails; show recipients and exact text; send only after approval; record status.

Social publishing

Prepare channel-specific posts; review images/captions; check supported API permissions; request approval; schedule; verify platform result.

Business phone call

Show purpose, number, script boundaries, language and estimated cost; obtain explicit authorization; disclose AI identity; honor recording/consent rules; summarize result.

Research report

Collect sources; distinguish evidence from speculation; generate cited report; save file and task record; offer follow-up actions.

5. Architecture and data flow

Primary path: desktop microphone -> audio capture/VAD -> streaming ASR -> conversation/session manager -> LLM + tool orchestrator -> permission gateway -> tool adapter -> outcome verifier -> memory and activity store -> streaming TTS and UI. Wake-word and interruption are handled by a dedicated audio controller; the assistant must stop speaking when the user interrupts and cancel or pause pending actions appropriately.

Recommended initial stack

Layer	Proposed choice	Why
Desktop UI	React + TypeScript + Tauri	Lightweight Windows desktop interface, task dashboard and audio controls.
Backend	Python + FastAPI + Pydantic	Strong AI ecosystem, typed tools and streaming endpoints.
Agent runtime	Start with explicit tool loop; add LangGraph as complexity grows	Understand agent fundamentals before adopting heavy orchestration.
Speech input	Benchmark Whisper-family, Deepgram and comparable services	Measure accented English accuracy, latency and cost with consented samples.
Speech output	Benchmark streaming cloud TTS and local fallback	Natural British-style voice, latency and pricing.
LLM	Replaceable provider adapter	Avoid lock-in; measure reasoning, tool reliability and cost.
Database	PostgreSQL; pgvector when retrieval is justified	Structured task data, history and semantic search.
Queue	Simple local jobs initially; Redis-backed worker later	Scheduled and long-running tasks with retry.
Browser	Playwright in isolated browser profile	Auditable multi-step web automation.
Desktop	Explicit allowlisted OS adapters	Constrain application launches, window and file operations.
Integrations	Official APIs and OAuth where available	Stable, revocable access and compliance.

Service boundaries

- Voice service: capture, wake word, VAD, ASR, TTS, barge-in, audio device management.
- Conversation service: prompt construction, personality modes, turn history and model streaming.
- Agent service: planning, tool selection, action graph, checkpoints, cancellation and verification.
- Permission service: capability scopes, approvals, sensitive-action policies and audit records.
- Tool adapters: OS, filesystem, browser, document generation, developer tooling, email, social, telephony.
- Memory service: structured preferences, conversation summaries, project/contact records, retrieval and deletion.
- Scheduler: periodic briefings, approved recurring jobs, reminders and follow-up monitoring.
- UI: live transcription, speech status, pending approvals, task timeline, settings and cost dashboard.

Core data model

Entity	Representative fields
UserSettings	voice, honorific, personality mode, languages, notification settings
Conversation	session ID, timestamps, turns, consent and retention
Memory	type, text, provenance, approval, sensitivity, expiration
ToolExecution	tool, arguments summary, permission decision, outcome, evidence, duration
Task	objective, plan, steps, owner, status, retries, artifacts
Contact	identity, channel handles, approved notes, retention rules
Integration	provider, granted scopes, token reference, connection health
ScheduledJob	trigger, task, approval policy, next run, status
CallRecord	authorization, AI disclosure, recording consent, transcript policy

6. Voice design and measurable quality

Recognition benchmark

- Create a private evaluation set of at least 50 consented commands spoken naturally in your accent, including application names, contacts and technical vocabulary.
- Test quiet-room and moderate-noise conditions; record word error rate, intent accuracy and false activations.
- Evaluate Tunisian Darija separately with code-switching examples before claiming support.
- Compare streaming and batch recognition; provide editable transcript before consequential actions.

Conversation behavior

- Aim for responsive first audio, but establish latency targets only after baseline hardware/network measurement.
- Implement wake-word false-positive/false-negative tests; visible microphone-active indicator and mute switch.
- Use barge-in to stop TTS immediately; define what happens to running tasks and confirmation dialogs.
- Configure a British-style butler voice; avoid unauthorized cloning of a real person.
- Personality: royal honorifics, dry humor, professional/casual/silent modes; never sacrifice factual accuracy for theatrical language.

7. Agent execution and safety specification

Action lifecycle

- 1. Parse objective and clarify ambiguous recipients, files, costs or irreversible actions.
- 2. Form a bounded plan and select tools from a registered, typed catalog.
- 3. Classify each proposed step by risk and required permissions.
- 4. Obtain approval for external messages, public posts, phone calls, purchases, destructive changes and other configured sensitive actions.

- 5. Execute with least privilege, bounded retries, timeouts and cancellation.
- 6. Verify the actual outcome using tool response and, where appropriate, a follow-up check.
- 7. Log evidence, summarize results and clearly label partial failures.

Permission policy

Class	Examples	Default
Read-only	Read approved folder, inspect issue, search public web	Allow within granted scope
Low-risk local	Open allowlisted app, draft a document	Allow or confirm per user setting
External write	Send message, publish post, open PR	Preview and explicit approval
Sensitive	Delete files, modify production data, paid call	Separate explicit confirmation
Prohibited by design	Bypass access control, secretly record, evade platform controls	Do not execute

Prompt-injection defense: treat email bodies, web pages, documents, repositories and transcripts as untrusted data. External text cannot override system policy, change tool permissions, request credentials or silently authorize actions. Enforce permissions in code outside the model.

8. External integrations and feasibility

Email

- Start with one provider using OAuth and minimal scopes; support inbox search, summarization, drafts, approved send, thread state and attachment handling.
- Use recipient verification, duplicate-send prevention, rate limits and visible send receipts.

Messaging and social

- Telegram: evaluate Bot API limitations versus personal messaging; do not promise universal access to private conversations.
- WhatsApp: assess supported Business Platform account requirements, templates, conversation windows and regional availability.
- Instagram: assess supported professional-account messaging/publishing APIs and app review requirements.
- Other platforms: add provider-specific adapters only after verifying supported scopes and policies.
- Content pipeline: idea -> draft -> media -> review -> schedule -> publish -> verify -> metrics; recurring posting requires user-configured authorization and a pause switch.

Business calling

- Use an authorized telephony provider only where AI-generated outbound calls and destination routes are supported.
- Require per-call recipient, purpose, cost limit, AI disclosure, and compliance with applicable calling and recording rules.
- Prefer English-language international businesses for initial pilots; Darija telephony is a research milestone, not a launch promise.
- Separate live transcript from post-call summary; configure retention and deletion, and record only with appropriate consent.

9. Detailed delivery phases

Phase 0 - Discovery and project foundation

Objective: Set a realistic budget, benchmark the environment and establish a testable architecture.

Implementation work

- Create monorepo, Python environment, Tauri/React shell, FastAPI service, linting, formatting, tests and CI.
- Document secrets handling, local file permissions, initial model provider and monthly spend cap.
- Design task/event schemas and mock tool adapter; implement basic telemetry and error handling.
- Create issue backlog and a small fixed set of acceptance scenarios.

Definition of done

- Fresh Windows setup works from README.
- CI passes; no credentials committed; local backend and desktop shell communicate.

Learning outcomes: Git, Python packaging, TypeScript, API design and basic testing.

Phase 1 - Voice-first conversational MVP

Objective: Hold natural, useful spoken English conversations with personality modes.

Prerequisites: Phase 0

Implementation work

- Benchmark ASR on accented-English sample set; build microphone streaming and VAD.
- Integrate LLM streaming and British-style streaming TTS.
- Create royal-butler personality with professional, casual and silent modes.
- Display live transcript, audio status, text fallback and conversation history.
- Measure latency, transcription errors and per-conversation cost.

Definition of done

- Speech input/output completes 20 consecutive test conversations without application crash.
- All three modes demonstrably change behavior; silent mode produces no speech.
- Transcription can be corrected before sensitive actions.

Learning outcomes: Speech pipelines, real-time streaming, prompt design and evaluation.

Phase 2 - Desktop tools, wake word and memory

Objective: Make JARVIS complete its first verifiable local tasks.

Prerequisites: Phase 1

Implementation work

- Implement tool registry, allowlisted application launcher and window management.

- Implement selected-folder file search, organization previews and summarization.
- Add wake-word detection and natural interruption with TTS cancellation.
- Persist approved preferences, recent tasks and searchable execution history.
- Build approval UI, audit log, task status and action verification.

Definition of done

- Voice command opens approved app and verifies it launched.
- Folder summary cites source files; file moves show preview and require configured approval.
- Wake-word and barge-in tests pass against recorded scenarios.
- User can inspect, correct and delete stored memories.

Learning outcomes: Tool calling, OS integration, memory, access control and observability.

Phase 3 - Developer and research agent

Objective: Make JARVIS useful for coding and multi-step research.

Prerequisites: Phase 2

Implementation work

- Add repository-scoped read/search, isolated test execution and diagnostic reports.
- Draft code patches; require approval before applying to protected branches.
- Integrate GitHub issues, branch creation and draft pull requests with minimal OAuth scopes.
- Add process/task monitoring with notifications and cancellation.
- Implement cited web research and Playwright workflows in a separate browser profile.

Definition of done

- Agent reproduces a seeded bug and produces a useful test report.
- Draft PR contains a diff and test evidence; no unauthorized push occurs.
- Research report includes inspectable sources and separates verified claims from inference.

Learning outcomes: Agent planning, sandboxing, browser automation, developer tooling and source evaluation.

Phase 4 - Communications and content

Objective: Support reliable human-approved outbound correspondence and publishing.

Prerequisites: Phase 3

Implementation work

- Add email inbox summaries, draft/reply workflows and approved send.
- Implement provider adapters for supported messaging channels and contact-specific context.
- Add post drafting, channel adaptation, review queue, scheduling and publication receipts.
- Use OAuth, token refresh, per-platform capability flags and duplicate-send protection.
- Build communication timeline and retention controls.

Definition of done

- Can draft and send a test email after explicit approval with verified receipt.
- Unsupported messaging capabilities are clearly marked unavailable.
- Approved scheduled post has visible queue status and verifiable provider response.

Learning outcomes: OAuth, API integration, background jobs, webhooks and rate limits.

Phase 5 - Chief-of-staff workflows

Objective: Coordinate projects, schedules and artifacts across connected services.

Prerequisites: Phase 4

Implementation work

- Create project/deadline/commitment tracker with task dependencies and reminders.
- Integrate an authorized calendar and produce morning/evening briefings.
- Suggest next actions with rationale and distinguish recommendations from scheduled commitments.
- Generate/edit DOCX/PDF and XLSX documents with previews and user-approved writes.
- Implement robust multi-step browser tasks with checkpoints and recovery.
- Add search over completed tasks, contact histories and generated artifacts.

Definition of done

- Briefing reflects real connected calendar and task state.
- Generated document and spreadsheet pass file integrity checks.
- Long-running workflow resumes after recoverable failure without duplicate external actions.

Learning outcomes: Workflow orchestration, scheduling, document automation and resilient state.

Phase 6 - Advanced social, multilingual voice and telephony

Objective: Extend the assistant only where platform access and quality benchmarks justify it.

Prerequisites: Phases 1-5

Implementation work

- Evaluate supported WhatsApp and Instagram business integrations, required account types and review process.
- Run separate Darija/code-switching ASR benchmark and improve recognition only with permitted data.
- Prototype authorized outbound business calls with AI disclosure and explicit per-call approval.
- Build compliant transcript and follow-up summary flow, retention controls and call cost guardrails.
- Stress-test interruption, multi-turn calls, failures, network loss and handoff to human.

Definition of done

- No unsupported platform access is advertised.
- A consented pilot call completes with correct disclosure, approved scope and usable summary.

- Darija support is labeled experimental until measured quality threshold is achieved.

Learning outcomes: Multilingual ASR, telephony, compliance engineering and evaluation.

Phase 7 - Production hardening and release

Objective: Deliver a maintainable, secure and observable single-user release.

Prerequisites: All preceding phases

Implementation work

- Perform threat modeling, dependency scans, secrets rotation and permission review.
- Create backup/restore, migration, update and rollback procedures.
- Add integration tests, end-to-end voice tests, failure-injection tests and cost monitoring.
- Package signed Windows desktop builds where practical; write installation and operations documentation.
- Run supervised beta, fix high-severity defects and publish release notes and limitations.

Definition of done

- Critical security issues resolved; acceptance suite passes.
- Recovery from database backup and revoked integrations tested.
- Users can stop jobs, revoke accounts and delete stored memories and transcripts.
- Installation and support instructions reproduce on a clean machine.

Learning outcomes: Security testing, deployment, maintenance and product release.

10. Suggested repository structure

```
jarvis/
  apps/desktop/                React + Tauri UI
  services/api/                FastAPI routes and auth
  services/voice/              ASR, VAD, wake word, TTS
  services/agent/              model adapters, planner, tool registry
  services/workers/            scheduled and long-running jobs
  packages/contracts/          shared typed schemas
  integrations/desktop/        allowlisted OS operations
  integrations/browser/        Playwright adapter
  integrations/github/         repo, issues, PR adapters
  integrations/email/          provider-specific mail adapters
  integrations/social/         supported channel adapters
  integrations/telephony/      consented call adapter
  storage/migrations/          database migrations
  tests/unit/                  tool and policy tests
  tests/integration/           provider mock tests
  tests/e2e/                   voice-to-action scenarios
  evals/speech/                consented audio benchmark metadata
  evals/agent/                 task success and safety cases
```

docs/	architecture, policies, runbooks

11. API contracts and operational design

Illustrative backend endpoints

Endpoint	Responsibility
POST /sessions	Create conversation session
WS /sessions/{id}/stream	Audio/text events and streamed responses
POST /tasks	Create bounded agent task
GET /tasks/{id}	Status, steps, evidence and artifacts
POST /tasks/{id}/approve	Approve exact proposed action
POST /tasks/{id}/cancel	Cancel task and pending tool execution
GET /integrations	Connection and capability status
POST /memories	Store explicit approved preference
GET /activity	Search execution history

Use typed request/response models, correlation IDs, versioned tool schemas and structured error codes. Do not expose arbitrary shell execution as a general-purpose LLM tool. All task events should support replay for UI reconnection and debugging.

12. Testing and acceptance matrix

Area	Required verification
Voice	Accent test set, noise cases, 20-turn stability, wake-word false activations, interruption latency
Personality	Honorifics and modes consistent; no invented tool outcomes
Desktop	Allowlist enforcement, file-scope isolation, safe file previews, verification
Developer	Reproducible tests, isolated execution, patch review, GitHub permission boundaries
Browser	Checkpoint/resume, authentication boundaries, malicious-page prompt injection
Email/social	Draft review, duplicate-send prevention, unsupported-capability detection
Calls	AI disclosure, explicit approval, cost ceiling, recording consent, call termination
Memory	Provenance, correction/deletion, sensitive-data retention and retrieval relevance
Reliability	API timeout, provider outage, job restart, backup/restore, network loss
Security	OAuth scope review, prompt injection, dependency scanning, audit integrity

Metrics dashboard

- Voice: word error rate, intent success, false wake rate, interruption delay, time to first audio.

- Agent: end-to-end task success, false-success reports, approval bypass count (target zero), recovery rate.
- Integrations: API error rate, rate-limit incidence, duplicate-send count (target zero), delivery verification.
- Operations: task duration, cost per task, uptime, crash frequency, restore success.
- Quality: user corrections, memory retrieval precision, citations verified and acceptance scenarios passed.

13. Security, privacy, compliance and ethical boundaries

- Use local-first defaults for microphone and files when feasible; explain when audio or content goes to external providers.
- Require explicit scopes for folders, repositories, accounts and contacts; allow revocation at any time.
- Encrypt transport; protect tokens using secure OS facilities; avoid storing raw credentials in prompts or logs.
- Define retention periods for audio, transcripts, emails, contact records and call metadata; offer deletion and export.
- Disclose AI identity during outbound calls; check applicable calling, consent and recording requirements before each deployment jurisdiction.
- Never automate spam, harassment, deceptive impersonation or mass unsolicited outreach; respect platform terms and rate limits.
- Guard against prompt injection in retrieved pages, emails, code comments and documents.
- Require human review for public posts, messages and consequential external actions by default.
- Treat model-generated plans as proposals; enforce safety, permissions and spending limits in deterministic code.

14. Budget and infrastructure planning

Maintain a monthly budget worksheet for LLM input/output tokens, speech recognition minutes, speech synthesis characters, phone minutes, hosted databases, storage and provider subscriptions. Free tiers and pricing change; verify before implementation. Start with hosted intelligence and local orchestration; benchmark local alternatives only after the product is stable.

Cost-control mechanisms

- Per-task token/time budgets and maximum tool iterations.
- Provider adapter allows changing LLM, ASR and TTS without rewriting business logic.
- Cache safe research results and avoid resynthesizing unchanged audio.
- Cap call minutes, destinations, message volume and scheduled-job frequency.
- Show estimated cost before paid or unusually expensive workflows.

15. Release criteria and completion definition

A project of this scope is never feature-finished in the abstract. Version 1.0 is considered complete when all 26 inventory items are implemented, formally marked optional/experimental, or documented as unavailable due to external platform constraints; no item may silently disappear.

- All P0 and P1 features meet their test acceptance criteria.
- P2 features operate through supported integrations with explicit approvals and audit trails.

- P3 features are either validated in supervised pilots or labeled experimental/unavailable with documented reasons.
- Security and privacy tests pass; credential revocation, data deletion and emergency stop work.
- The system can recover from provider errors and report failures accurately.
- Installation, configuration, cost, architecture, API, test and user manuals are published.
- At least one full workflow is validated in each supported domain: voice, desktop, development, research, communication, productivity and telephony (if enabled).

16. Risks, dependencies and mitigation

Risk	Mitigation
Project abandonment due to scope	Weekly demos, one vertical slice per phase, no unplanned integrations before acceptance.
Speech accuracy for accent/Darija	Build private benchmark early; choose engine from measured results.
High API costs	Spending limits, model routing, caching and usage dashboard.
Platform API restrictions	Capability matrix, official APIs, documented unsupported operations.
Hallucinated task completion	Structured tool receipts and explicit verification.
Prompt injection or credential leakage	Deterministic permission gateway, least privilege, untrusted-content isolation.
Desktop automation fragility	Prefer APIs and deterministic adapters; add UI automation only when necessary.
Telephone compliance and quality	Jurisdiction checks, disclosure, consent, narrow pilots and call limits.
Limited development hardware	Hosted model option; lightweight desktop stack and asynchronous workers.

17. First seven-day implementation sprint

Day	Deliverable
1	Monorepo, FastAPI health endpoint, React/Tauri shell, environment templates and README.
2	Provider-agnostic LLM client and streamed text chat.
3	Royal-butler prompt and three personality modes with regression tests.
4	Microphone capture, ASR benchmark harness and first accent evaluation.
5	Streaming TTS and playback controls.
6	First allowlisted app-opening tool with explicit outcome verification.
7	Demo: spoken command -> transcript -> model -> approved tool -> verified spoken result.

18. Immediate next actions

- Confirm preferred monthly API budget, Windows setup, and whether cloud speech processing is acceptable.
- Create Git repository and issue board using the phases and feature IDs in this document.

- Implement Phase 0 and the first seven-day sprint before adding social or telephony providers.
- Record baseline accented-English commands and establish measurable speech quality.
- Maintain this document as the source of truth: change log, feature matrix, acceptance evidence and release status.

End of blueprint. This document is a planning specification; technology choices, pricing, legal requirements and platform availability must be verified at implementation time.